

USER MANUAL

Sistem Informasi Lost & Found IPB (FoundIt)

Mata Kuliah : Keamanan Informasi (KOM1315) **Semester :** Genap 2025/2026 **Program Studi :** Ilmu Komputer, IPB University **Jenis Dokumen :** Panduan Penggunaan Sistem (User Manual)
Kelompok 2 :

- Syafiq Syadidul Azmi (G6401231075)
 - Muhammad Faqih (G6401231081)
 - Luqman Fadillah Santoso (G6401231136)
-

DAFTAR ISI

1. [Pengenalan Sistem](#)
 2. [Prasyarat dan Instalasi](#)
 3. [Cara Menjalankan Sistem](#)
 4. [Panduan Pengguna — Registrasi dan Login](#)
 5. [Panduan Pengguna — Barang Hilang dan Temuan](#)
 6. [Panduan Pengguna — Klaim Kepemilikan](#)
 7. [Panduan Pengguna — Riwayat Aktivitas](#)
 8. [Panduan Pengguna — Notifikasi](#)
 9. [Panduan Administrator](#)
 10. [Referensi API \(Swagger UI\)](#)
 11. [Troubleshooting](#)
-

1. Pengenalan Sistem

FoundIt — Lost & Found IPB adalah platform berbasis web yang memfasilitasi pelaporan dan pencarian barang hilang/temuan di lingkungan kampus IPB University. Sistem ini dirancang

dengan pendekatan *security-by-design*, di mana protokol keamanan informasi ditanamkan sejak tahap arsitektur.

1.1 Fitur Unggulan

 Autentikasi Aman	Login berbasis JWT dengan masa berlaku terbatas
 Enkripsi Data Identitas	KTP dan NIK disimpan dalam bentuk terenkripsi
 Kontrol Akses Berbasis Peran	Tiga tingkat akses: UMUM, CIVITAS, ADMIN
 Klaim Terstruktur	Sistem klaim dengan bukti foto dan teks terverifikasi
 Notifikasi Otomatis	Pemberitahuan status klaim secara real-time
 Audit Trail	Seluruh aktivitas sistem dicatat untuk akuntabilitas

1.2 Peran Pengguna

UMUM	Default saat registrasi	Melihat daftar barang, registrasi
CIVITAS	Otomatis jika email @ipb.ac.id	Semua UMUM + laporkan barang, ajukan klaim
ADMIN	Ditentukan oleh administrator	Semua CIVITAS + review klaim, kelola user, audit log

2. Prasyarat dan Instalasi

2.1 Kebutuhan Sistem

Pastikan perangkat memiliki software berikut:

Docker	20.x	Container runtime
Docker Compose	2.x (plugin)	Orkestrasi container

Git	2.x	Version control
-----	-----	-----------------

Catatan: Docker Desktop sudah mencakup Docker dan Docker Compose dalam satu instalasi.

2.2 Instalasi Docker

Windows:

1. Unduh Docker Desktop dari <https://www.docker.com/products/docker-desktop/>
2. Jalankan installer dan ikuti wizard
3. Restart komputer setelah instalasi
4. Verifikasi: buka terminal dan jalankan `docker --version`

Verifikasi Instalasi:

```
docker --version
# Output: Docker version 25.x.x

docker compose version
# Output: Docker Compose version v2.x.x
```

3. Cara Menjalankan Sistem

3.1 Clone Repositori

```
git clone https://github.com/Didul-arch/KOM1315_SmtGenap26_Kelompok2_IPB-LostnFound.git
cd KOM1315_SmtGenap26_Kelompok2_IPB-LostnFound
```

3.2 Konfigurasi Environment

Masuk ke folder backend dan buat file `.env` dari template:

```
cd 03_Source_Code/backend
copy .env.example .env
```

Edit file `.env` dan sesuaikan nilai-nilainya:

```
# Konfigurasi Database
DATABASE_URL=postgresql+asyncpg://user:password@db:5432/lostnfound
POSTGRES_DB=lostnfound
POSTGRES_USER=user
POSTGRES_PASSWORD=password

# Keamanan – WAJIB DIGANTI untuk produksi
SECRET_KEY=ganti-dengan-random-256-bit-secret-yang-kuat
ENCRYPTION_KEY=ganti-dengan-fernet-key-32-byte-base64

# Konfigurasi Aplikasi
ALGORITHM=HS256
ACCESS_TOKEN_EXPIRE_MINUTES=60
CLAIM_UPLOAD_DIR=storage/claims
IDENTITY_DOCUMENT_UPLOAD_DIR=storage/identity

# CORS – Sesuaikan dengan URL frontend
CORS_ORIGINS=http://localhost:3000,http://localhost:5173
```

⚠ **Penting:** Jangan pernah commit file `.env` ke repositori. File ini sudah terdaftar di `.gitignore`.

Cara membuat ENCRYPTION_KEY yang valid:

```
# Jalankan di Python
from cryptography.fernet import Fernet
print(Fernet.generate_key().decode())
```

3.3 Menjalankan Aplikasi

Dari dalam folder `03_Source_Code`, jalankan:

```
cd 03_Source_Code
```

```
docker compose -f backend/docker-compose.yml up --build -d
```

Perintah ini akan:

1. Membangun image Docker backend (FastAPI)
2. Menjalankan container database PostgreSQL 17
3. Menjalankan container backend
4. Melakukan migrasi database secara otomatis (Alembic)

3.4 Verifikasi Sistem Berjalan

```
# Cek status container
docker ps

# Output yang diharapkan:
# NAMES                STATUS                PORTS
# ki_backend            Up X minutes          0.0.0.0:8000->8000/tcp
# postgres_db           Up X minutes          0.0.0.0:6969->5432/tcp
```

Buka browser dan akses:

- API Utama: <http://localhost:8000>
- Swagger Docs: <http://localhost:8000/docs>
- ReDoc: <http://localhost:8000/redoc>

Jika berhasil, `http://localhost:8000` akan menampilkan:

```
{"message": "API is running!"}
```

3.5 Seed Data Awal (Opsional)

Untuk mengisi database dengan data awal termasuk akun admin:

```
docker exec -it ki_backend python -m backend.app.seed
```

3.6 Menghentikan Sistem

```
docker compose -f backend/docker-compose.yml down
```

Untuk menghapus data database juga:

```
docker compose -f backend/docker-compose.yml down -v
```

4. Panduan Pengguna — Registrasi dan Login

4.1 Registrasi Akun Baru

Endpoint: `POST /api/v1/auth/users/` Content-Type: `multipart/form-data`

Langkah via Swagger UI (<http://localhost:8000/docs>):

1. Buka bagian **auth** → klik `POST /api/v1/auth/users/`
2. Klik **Try it out**
3. Isi field berikut:

<code>email</code>	✓	Email aktif. Gunakan <code>@ipb.ac.id</code> untuk peran CIVITAS
<code>fullname</code>	✓	Nama lengkap sesuai identitas
<code>password</code>	✓	Password minimal 8 karakter
<code>phone_number</code>	✓	Nomor telepon aktif
<code>identity_number</code>	✗	NIK (Nomor Induk Kependudukan) — akan dienkrpsi
<code>identity_document</code>	✗	File foto KTP/identitas — akan dienkrpsi dan disimpan aman

4. Klik **Execute**

5. Response sukses: `{"message": "User Berhasil Dibuat", "data": {...}}`

Keamanan: Password Anda disimpan dalam bentuk hash Argon2id — tidak dapat dibaca siapa pun termasuk administrator. NIK dan foto KTP disimpan terenkripsi menggunakan AES-128 (Fernet).

4.2 Login

Endpoint: `POST /api/v1/auth/login` Content-Type: `application/x-www-form-urlencoded`

Langkah via Swagger UI:

1. Buka `auth` → `POST /api/v1/auth/login`
2. Klik **Try it out**
3. Isi:
 - `username` : email yang didaftarkan
 - `password` : password Anda
4. Response sukses:

```
{
  "access_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "token_type": "bearer"
}
```

Catatan keamanan: `access_token` berlaku **60 menit**. Setelah habis, gunakan `refresh_token` untuk mendapatkan token baru tanpa login ulang.

4.3 Menggunakan Token di Swagger

1. Copy nilai `access_token`
2. Klik tombol **Authorize** (ikon gembok ) di bagian atas halaman Swagger
3. Paste token di field **Value** dengan format: `Bearer <token>`
4. Klik **Authorize** → **Close**

5. Seluruh endpoint yang memerlukan autentikasi kini dapat digunakan

4.4 Refresh Token

Jika access token kedaluwarsa (error `401`):

Endpoint: `POST /api/v1/auth/token/refresh`

```
{
  "refresh_token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9..."
}
```

Response: access token dan refresh token baru.

4.5 Melihat Profil Diri Sendiri

Endpoint: `GET /api/v1/users/me` (Memerlukan JWT)

Response:

```
{
  "id": 1,
  "email": "mahasiswa@apps.ipb.ac.id",
  "fullname": "Nama Lengkap",
  "phone_number": "08xxxxxxxxxx",
  "role": "CIVITAS",
  "is_active": true,
  "identity_number": "3201xxxxx",
  "identity_document": "/api/v1/users/1/identity-document"
}
```

5. Panduan Pengguna — Barang Hilang dan Temuan

5.1 Melihat Daftar Barang

Endpoint: `GET /api/v1/items/` (Tidak perlu login)

Semua pengunjung dapat melihat daftar barang yang dilaporkan.

Response:

```
{
  "data": [
    {
      "id": 1,
      "title": "Dompel Hitam",
      "description": "Dompel kulit hitam dengan isi KTM",
      "location": "Gedung FMIPA Lt. 2",
      "report_type": "lost",
      "status": "NOT_RETURNED",
      "created_at": "2026-06-08T10:00:00",
      "image": "/storage/items/abc123.jpg"
    }
  ]
}
```

5.2 Mencari Barang

Endpoint: `GET /api/v1/items/search`

Parameter filter yang tersedia:

<code>report_type</code>	string	<code>lost</code> atau <code>found</code>	Jenis laporan
<code>location</code>	string	FMIPA	Lokasi kejadian (partial match)
<code>report_date</code>	date	2026-06-08	Tanggal pelaporan (format YYYY-MM-DD)

5.3 Melaporkan Barang Hilang

Syarat: Akun CIVITAS (email `@ipb.ac.id` atau `@apps.ipb.ac.id`) + sudah login

Endpoint: `POST /api/v1/items/report-lost` **Content-Type:** `multipart/form-data`

<code>title</code>	✓	Nama/judul barang yang hilang

description	✓	Deskripsi detail barang
location	✓	Lokasi terakhir barang terlihat
category	✗	Kategori barang (elektronik, dokumen, dll.)
image	✗	Foto barang (jpg, png, heic)

Contoh pengisian:

- title : Laptop Asus ROG Hitam
- description : Laptop gaming, stiker bendera merah putih di tutup, charger tertinggal
- location : Kantin Botani Square, meja pojok kiri
- category : elektronik

5.4 Melaporkan Barang Temuan

Syarat: Akun CIVITAS + sudah login

Endpoint: POST /api/v1/items/report-found Content-Type: multipart/form-data

Sama seperti form laporan hilang, namun melaporkan barang yang **Anda temukan**.

Tips pelaporan yang baik:

- Cantumkan ciri khas yang spesifik pada description
- Sertakan foto jika memungkinkan
- Pilih lokasi yang detail (gedung, lantai, ruangan)

6. Panduan Pengguna — Klaim Kepemilikan

6.1 Mengajukan Klaim

Syarat: Akun CIVITAS + sudah login + memiliki bukti kepemilikan

Untuk mengklaim barang yang Anda yakini milik Anda:

Endpoint: `POST /api/v1/claims/` **Content-Type:** `multipart/form-data`

<code>item_id</code>	✓	ID barang yang ingin diklaim (lihat dari daftar barang)
<code>proof_text</code>	✓	Deskripsi bukti kepemilikan secara detail
<code>proof_image</code>	✓	Foto bukti (misalnya foto bersama barang tersebut sebelumnya)
<code>request_type</code>	✗	<code>claim</code> (default) untuk klaim kepemilikan

Panduan `proof_text` yang kuat:

- Sebutkan nomor seri, warna, merek, atau tanda khusus
- Ceritakan kapan dan di mana terakhir Anda menggunakan barang tersebut
- Jelaskan isi dalam tas/dompet jika ada

6.2 Melihat Status Klaim

Endpoint: `GET /api/v1/claims/` (Memerlukan JWT)

User non-admin hanya dapat melihat klaim milik sendiri.

```
{
  "data": [
    {
      "id": 5,
      "request_type": "claim",
      "status": "PENDING",
      "proof_text": "Laptop milik saya dengan stiker ...",
      "item_id": 12,
      "created_at": "2026-06-08T14:00:00"
    }
  ]
}
```

Status klaim:

- **PENDING** — Menunggu review admin
- **APPROVED** — Klaim disetujui, silakan ambil barang
- **REJECTED** — Klaim ditolak

6.3 Mengambil Barang (Mark as Collected)

Setelah klaim Anda disetujui (**APPROVED**), tandai barang sudah diambil:

Endpoint: `PATCH /api/v1/claims/{claim_id}/mark-collected`

Ini akan mengubah status item menjadi **RETURNED** .

7. Panduan Pengguna — Riwayat Aktivitas

7.1 Melihat Riwayat

Endpoint: `GET /api/v1/history/me` (Memerlukan JWT)

Menampilkan seluruh riwayat aktivitas Anda — barang yang pernah dilaporkan dan klaim yang pernah diajukan.

```
{
  "data": [
    {
      "kind": "report",
      "title": "Dompot Hitam",
      "location": "FMIPA",
      "item_status": "NOT_RETURNED",
      "created_at": "2026-06-01T09:00:00"
    },
    {
      "kind": "request",
      "request_type": "claim",
      "request_status": "APPROVED",
      "title": "Laptop Asus",
      "can_mark_returned": true
    }
  ]
}
```

7.2 Menandai Barang Dikembalikan

Jika Anda adalah pelapor kehilangan (bukan pengklaim) dan barang sudah ditemukan/dikembalikan:

Endpoint: `PATCH /api/v1/history/items/{item_id}/returned`

8. Panduan Pengguna — Notifikasi

8.1 Melihat Notifikasi

Endpoint: `GET /api/v1/notifications/` (Memerlukan JWT)

Sistem otomatis mengirimkan notifikasi saat:

- Ada pengguna yang mengajukan klaim atas barang yang Anda laporkan
- Status klaim Anda diubah oleh admin (disetujui/ditolak)

```
{
  "data": [
    {
      "id": 1,
      "message": "Ada klaim baru untuk barang 'Dompel Hitam' Anda",
      "is_read": false,
      "created_at": "2026-06-08T15:00:00"
    }
  ]
}
```

9. Panduan Administrator

9.1 Cara Masuk sebagai Admin

Admin menggunakan endpoint login yang sama, namun akun harus memiliki `role: ADMIN`. Akun admin dapat dibuat melalui seed data atau diubah perannya oleh admin lain.

9.2 Mereview Klaim

Endpoint: `PATCH /api/v1/claims/{claim_id}/review` Body (JSON):

```
{
  "status": "APPROVED"
}
```

Nilai `status` yang valid: `APPROVED` atau `REJECTED`

Alur review yang disarankan:

1. Lihat daftar klaim PENDING: `GET /api/v1/claims/?status=PENDING`
2. Buka detail klaim tertentu: `GET /api/v1/claims/{id}`
3. Periksa `proof_text` dan `proof_image`
4. Kirim keputusan review

Setelah review, sistem otomatis mengirimkan notifikasi ke pengklaim.

9.3 Manajemen Pengguna

Melihat semua pengguna:

```
GET /api/v1/users/
```

Melihat detail lengkap pengguna (termasuk data sensitif yang didekripsi):

```
GET /api/v1/admin/users/{user_id}
```

Mengubah peran pengguna:

```
PATCH /api/v1/users/{user_id}
Body: { "role": "ADMIN" } (form-data)
```

Menonaktifkan pengguna:

```
PATCH /api/v1/users/{user_id}
Body: { "is_active": false }
```

Menghapus pengguna:

```
DELETE /api/v1/users/{user_id}
```

9.4 Melihat Dokumen Identitas

Admin dapat melihat dokumen identitas pengguna (KTP) yang secara otomatis didekripsi:

Endpoint: `GET /api/v1/users/{user_id}/identity-document`

Response: File gambar KTP dalam format JPEG (streaming response, didekripsi on-the-fly dari file `.enc` terenkripsi).

9.5 Audit Log

Endpoint: `GET /api/v1/admin/audit-logs?page=1&limit=15`

Menampilkan seluruh riwayat aktivitas sistem dengan paginasi:

```
{
  "data": [
    {
      "id": 1,
      "user_id": 5,
      "action": "POST /api/v1/auth/login",
      "endpoint": "/api/v1/auth/login",
      "ip_address": "192.168.1.100",
      "user_agent": "Mozilla/5.0 ...",
      "status_code": 200,
      "created_at": "2026-06-08T21:00:00"
    }
  ],
  "total": 250,
  "page": 1,
  "limit": 15
}
```

page	1	-	Nomor halaman
limit	15	100	Jumlah data per halaman

10. Referensi API (Swagger UI)

Seluruh endpoint dapat dijelajahi dan dicoba langsung melalui **Swagger UI**:

URL: <http://localhost:8000/docs>

Fitur Swagger UI:

-  Dokumentasi otomatis semua endpoint
-  Form autentikasi JWT terintegrasi
-  "Try it out" untuk mencoba langsung
-  Upload file langsung dari browser

Cara menggunakan autentikasi di Swagger:

1. Login via `POST /api/v1/auth/login`
2. Copy `access_token` dari response
3. Klik tombol **Authorize** () di pojok kanan atas
4. Masukkan: `Bearer eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...`
5. Klik **Authorize**

11. Troubleshooting

Masalah Umum dan Solusi

<code>401 Unauthorized</code>	Token tidak valid atau kadaluwarsa	Login ulang atau gunakan refresh token
<code>403 Forbidden</code>	Tidak memiliki hak akses	Pastikan peran user sesuai kebutuhan endpoint
<code>422 Unprocessable Entity</code>	Format data salah	Periksa tipe data dan field wajib
<code>429 Too Many Requests</code>	Rate limit login terlampaui (5x/menit)	Tunggu 1 menit sebelum mencoba lagi

Container tidak mau start	Port sudah dipakai	Cek <code>docker ps</code> dan hentikan container lain di port 8000/6969
Database error saat start	Migrasi gagal	Jalankan <code>docker compose down -v</code> lalu ulang <code>up --build</code>
<code>.env</code> tidak terbaca	File tidak ada di folder <code>backend/</code>	Copy dari <code>.env.example</code> dan isi nilainya

Log dan Debugging

Melihat log backend:

```
docker logs ki_backend -f
```

Melihat log database:

```
docker logs postgres_db -f
```

Masuk ke shell container backend:

```
docker exec -it ki_backend bash
```

Cek koneksi database:

```
docker exec -it postgres_db psql -U user -d lostnfound -c "\dt"
```

Dokumen ini merupakan panduan penggunaan sistem Lost & Found IPB untuk mata kuliah Keamanan Informasi (KOM1315), Program Studi Ilmu Komputer, IPB University, Semester Genap 2025/2026.